SRD Benchmark — K-Quant Rerun

Google Colab step-by-step guide · llama.cpp pre-built binaries

What this guide does

The first SRD benchmark run used **cited** K-quant numbers from the llama.cpp README (Q4_K_M, Q5_K_M, Q6_K, Q8_0). To make the comparison airtight — same model, same dataset split, same stride — we need to measure those numbers locally using llama.cpp's own `llama-perplexity` tool.

This guide avoids building llama.cpp from source (the main Colab failure point). It downloads a pre-built Ubuntu binary from the llama.cpp GitHub releases page instead. Total runtime on a T4 is roughly **25–40 minutes**.

> *Prerequisite: you already have research/quant/results/srd_sweep.json from the first Colab run. This guide adds kquant_rerun.json alongside it and regenerates the plot.*

1 Open a GPU runtime

In the Colab menu: **Runtime** → **Change runtime type** → **T4 GPU**. Then run the cell below to confirm the GPU is visible and note the CUDA version — you need this to pick the right binary in Step 2.

```
!nvidia-smi
!nvcc --version | grep 'release'
```

Cell 1

Expected output: a table showing the T4 with ~15 GB VRAM, and a line like `release 12.x`*. If you see No GPU found, the runtime type wasn't saved — repeat the menu step and reconnect.*

2 Download pre-built llama.cpp binaries

■ Do NOT build from source on Colab. The CUDA toolkit version on Colab frequently mismatches what llama.cpp's cmake expects, causing silent CPU-fallback builds or outright failures. Pre-built binaries are faster and reliable.

The cell below fetches the latest release tag from the GitHub API, downloads the matching Ubuntu CUDA binary zip, and installs the two binaries we need (`llama-perplexity` and `llama-quantize`):

```
import urllib.request, json, zipfile, os, stat, pathlib                    Cell 2

# Get latest release tag
api = 'https://api.github.com/repos/ggerganov/llama.cpp/releases/latest'
with urllib.request.urlopen(api) as r:
    tag = json.load(r)['tag_name']   # e.g. 'b5012'
print('Latest llama.cpp release:', tag)

# Download Ubuntu CUDA binary zip
zip_name = f'llama-{tag}-bin-ubuntu-x64.zip'
url = (f'https://github.com/ggerganov/llama.cpp'
       f'/releases/download/{tag}/{zip_name}')
print('Downloading', zip_name, '...')
urllib.request.urlretrieve(url, '/tmp/llama.zip')

# Extract and make binaries executable
os.makedirs('/opt/llama_bin', exist_ok=True)
with zipfile.ZipFile('/tmp/llama.zip') as z:
    for name in z.namelist():
        if name.endswith(('llama-perplexity', 'llama-quantize',
                          'llama-perplexity.exe',
                          'llama-quantize.exe')):
            z.extract(name, '/opt/llama_bin')
            full = pathlib.Path('/opt/llama_bin') / name
            full.chmod(full.stat().st_mode | stat.S_IEXEC)

# Verify
!ls -lh /opt/llama_bin/
!/opt/llama_bin/llama-perplexity --version 2>&1 | head -2
```

If the download fails with a 404, the release zip may use a slightly different name (some releases append `-cuda12`). Check the assets list on the releases page and update `zip_name` accordingly.

3 Get convert-hf-to-gguf.py and install Python deps

The conversion script lives in the llama.cpp repo root (not in the binary zip). A shallow clone is fast — we only need the script, not the full history.

```
!git clone --depth 1 https://github.com/ggerganov/llama.cpp /opt/llama_src      Cell 3

# Python deps for the convert script + our research harness
!pip install -q transformers datasets sentencepiece protobuf
!pip install -q torch --index-url https://download.pytorch.org/whl/cu121

# Clone the Axiom repo (skip if already mounted via Drive)
!git clone --depth 1 https://github.com/orivael-dev/axiom /content/axiom
%cd /content/axiom
```

If you already have the Axiom repo on Drive, mount it with `from google.colab import drive; drive.mount('/content/drive')` and `%cd /content/drive/MyDrive/axiom` instead of the clone.

4 Save the WikiText-2 test split

llama-perplexity reads the eval text from a plain file. We use the same HuggingFace dataset split our SRD sweep used, joined with double newlines — the standard WikiText convention.

Cell 4

```
from datasets import load_dataset

ds = load_dataset('wikitext', 'wikitext-2-raw-v1', split='test')
text = '\n\n'.join(ds['text'])

wikitext_path = '/tmp/wikitext2_test.txt'
with open(wikitext_path, 'w') as f:
    f.write(text)

print(f'Saved {len(text):,} chars to {wikitext_path}')
```

5 Run the K-quant rerun benchmark

This cell runs `bench_llamacpp.py` in `--rerun-locally` mode. It will:

1. Download TinyLlama from HuggingFace (~2 GB).
2. Convert to GGUF F16 using `convert-hf-to-gguf.py`.
3. Quantize to Q4_K_M, Q5_K_M, Q6_K, Q8_0 using `llama-quantize`.
4. Run `llama-perplexity` on each (~5–8 min each).

Cell 5

```
import subprocess, sys

result = subprocess.run([
    sys.executable, '-m', 'research.quant.bench_llamacpp',
    '--rerun-locally',
    '--llama-bin',      '/opt/llama_bin',
    '--llama-src',      '/opt/llama_src',
    '--wikitext-file',  '/tmp/wikitext2_test.txt',
    '--gguf-dir',       '/tmp/srd_gguf',
    '--output',         'research/quant/results/kquant_rerun.json',
], check=False)

if result.returncode != 0:
    print('benchmark exited with code', result.returncode)
else:
    print('Done — results in research/quant/results/kquant_rerun.json')
```

■ Note on stride: llama-perplexity defaults to stride = context (2048), while the SRD sweep used stride 512. Our results file flags this as 'rerun_local'. A small PPL difference (~0.05–0.1) between methods is expected and does not affect the 1.51 PPL gap finding.

6 Regenerate the plot with local numbers

Once the rerun JSON exists, regenerate the scatter plot so it shows locally-measured K-quant points instead of cited ones. The orange SRD squares don't change — only the teal K-quant line updates.

```
!python -m research.quant.plot_results \
  --inputs research/quant/results/srd_sweep.json,\
           research/quant/results/kquant_rerun.json \
  --output  docs/srd_perplexity_vs_bpw.png

# Display inline
from IPython.display import Image
Image('docs/srd_perplexity_vs_bpw.png')
```

Cell 6

7 Download results and push

Download both result files to your machine, or commit them directly from Colab.

```
from google.colab import files

files.download('research/quant/results/kquant_rerun.json')
files.download('docs/srd_perplexity_vs_bpw.png')
```

Cell 7a — download

Or push straight to the branch:

```
!git config user.email 'you@example.com'
!git config user.name  'Your Name'
!git add research/quant/results/kquant_rerun.json \
         docs/srd_perplexity_vs_bpw.png
!git commit -m 'results: K-quant local rerun on Colab T4'
!git push origin claude/srd-prototype-benchmark-JRtv1
```

Cell 7b — push

Troubleshooting

Cell 2: 404 downloading the zip

Some releases use a different filename suffix. Open github.com/ggerganov/llama.cpp/releases/latest in a browser, look at the Assets list, find the Ubuntu x64 zip name, and replace zip_name in the cell with the exact filename shown.

Cell 2: llama-perplexity not found in the zip

Older releases named it llama-perplexity (no prefix) or perplexity. Print z.namelist() inside the with block to see what's actually in the zip, then update the endswith() check to match.

Cell 3: torch install takes too long

Skip the torch install if it's already present: python -c 'import torch' to check. Colab's base image usually has torch pre-installed.

Cell 5: convert-hf-to-gguf.py not found

The script expects --llama-src pointing at the repo root where the script lives. Confirm it exists: ls /opt/llama_src/convert-hf-to-gguf.py. If missing, the clone in Cell 3 may have failed — re-run it.

Cell 5: llama-perplexity CUDA error / falls back to CPU

This happens if the binary was compiled for a different CUDA version. Check nvcc --version (Cell 1) and confirm the binary release matches. If mismatched, pass --device cpu to bench_llamacpp.py (adds ~5× wallclock but still works).

Cell 5: TinyLlama download hangs

HuggingFace rate-limits anonymous downloads. Add --hf-token to the bench_llamacpp.py call, or set HF_TOKEN in the env before running.

Cell 5: llama-quantize: command not found

The binary may be nested inside a subdirectory in the zip. Run find /opt/llama_bin -name llama-quantize to locate it, then symlink it: ln -s /opt/llama_bin/llama-quantize

---

Expected timing on Colab T4

Step	Approx. time
Cell 2: download pre-built binaries	< 1 min
Cell 3: clone + pip install	3–5 min
Cell 4: save WikiText-2	< 1 min
Cell 5: GGUF conversion (F16)	3–5 min
Cell 5: llama-quantize × 4	2–4 min
Cell 5: llama-perplexity × 4 (GPU)	20–30 min
Cell 6: plot	< 1 min
Total	**~30–45 min**

After the run, paste kquant_rerun.json back into this session and the write-up will update automatically.